COMP0250 - Pick and Place, Object Detection and Localization

Group 13

Ramneek Ahluwalia Ethan Dsouza

1 Overview

1.1 Project Overview

This package implements solutions for the COMP0250 Pick and Place, Object Detection and Localization coursework. Our system is designed to perform robotic manipulation tasks in a simulated environment using ROS, MoveIt!, and PCL to detect object positions, orientations, and colors. Our solution addresses three main tasks:

- Task 1: Implementation of a robust pick-and-place system that can identify object orientation using Harris Corner detection and accurately grasp objects regardless of their positioning.
- Task 2: Development of an object detection and classification algorithm that can identify different shapes (noughts vs. crosses) using centroid analysis and point cloud processing.
- Task 3: Creation of an integrated planning and execution system that combines techniques from Tasks 1 and 2 to identify multiple objects, classify them, and execute appropriate pick-and-place operations based on object properties.

Key Innovations:

1. Advanced Object Orientation Detection: Our solution uses Harris Corner detection to precisely identify object orientation, allowing for accurate grasping regardless of how objects are positioned in the workspace.
2. Centroid-Based Classification: We developed a novel approach to object classification by analyzing the distribution of points near the centroid, enabling reliable differentiation between similar shapes.
3. Safety-Optimized Path Planning: For complex environments with obstacles, our system calculates paths that maximize distance from obstacles, enhancing operational safety.
4. Comprehensive Workspace Scanning: Our solution implements a systematic scanning approach that covers both front and rear areas of the workspace, ensuring complete environment visibility.
5. Adaptive Object Handling: The system can identify and process different object types (noughts, crosses, obstacles, boxes) and adjust its manipulation strategy accordingly.

The implementation demonstrates strong performance across all three tasks, with particular attention to robustness against edge cases and environmental variations. Our approach prioritises collision avoidance, efficient execution and accurate object classification.

1.2 Task Breakdown and Contribution

For this project, the tasks were completed as follows:

- Task 1 - (50% Ethan, 50% Ramneek) - 25 hours total
- Task 2 - (50% Ethan, 50% Ramneek) - 4 hours total
- Task 3 - (50% Ethan, 50% Ramneek) - 15 hours total

2 Package Setup

To build our package simply run the following:
`catkin build cw2_team_13`
Then to run our solution execute the ros launch command:
`roslaunch cw2_team_13 run_solution.launch`

Occasionally, you may encounter an issue where the system fails to read point cloud data. If you see related error messages in the console during testing, please try either restarting the current task or re-executing the roslaunch command to resolve the problem.

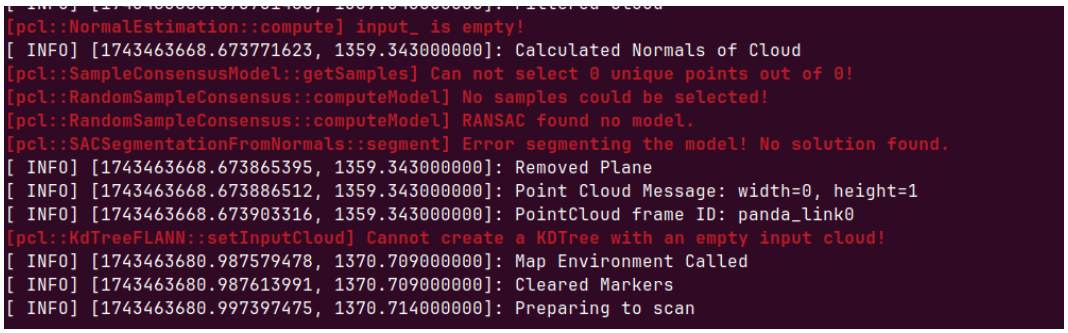

Figure 1: Output from our point cloud node

We've observed that running the tasks in a particular sequence can sometimes cause environmental inconsistencies. Specifically, if you execute Task 2 before Task 3, the ground plane may occasionally be positioned higher than expected, causing Task 3 objects to appear embedded within the surface. If you encounter this issue, please restart the system by re-running the roslaunch command, and then execute Task 3 before Task 2 to ensure proper object placement.

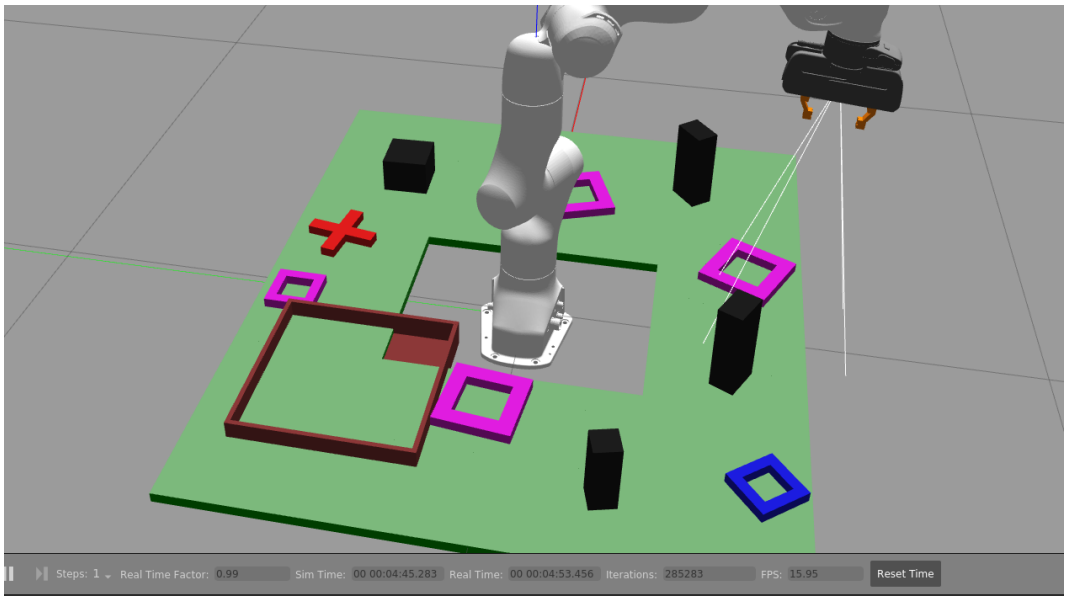

Figure 2: Gazebo Task 3 spawning bug example

3 Task 1

Our approach to Task 1 begins with a thorough scan of the workspace's front area to accurately determine object orientation, which is crucial for precise grasping. After identifying the object, we calculate the centroid of the object(x_c, y_c) and strategically position the gripper at a calculated offset point to the left of the object (x_1, y_1). Using transformation equations, we compute the optimal grasping position that accounts for the object's orientation. Once the object is securely grasped, we navigate to the goal location specified in the service request and carefully deposit the object at its destination using the RRTStar planner from MoveIt.
(Note: Since our implementation maintains a fixed x-axis orientation, we can simplify the transformation equations by omitting the first term from each equation, streamlining the calculation process.)

$$x_1 = (x_0 - x_c)\cos(\theta) - (y_0 - y_c)\sin(\theta) + x_c \tag{1}$$
$$y_1 = (x_0 - x_c)\sin(\theta) + (y_0 - y_c)\cos(\theta) + y_c \tag{2}$$

We determine the orientation angle θ using the Harris Corner detection algorithm to identify key points on the object. Our approach varies by object type: For square objects, we analyse points along the y-axis to locate the lowest point, then calculate the orientation using this point and its nearest corner to the right. For cross-shaped objects, we identify the two lowest corners that are closest to the centroid and use these to determine the orientation angle. For visualisation purposes, object orientations are displayed in RViz by default, allowing for real-time verification of the detection process.

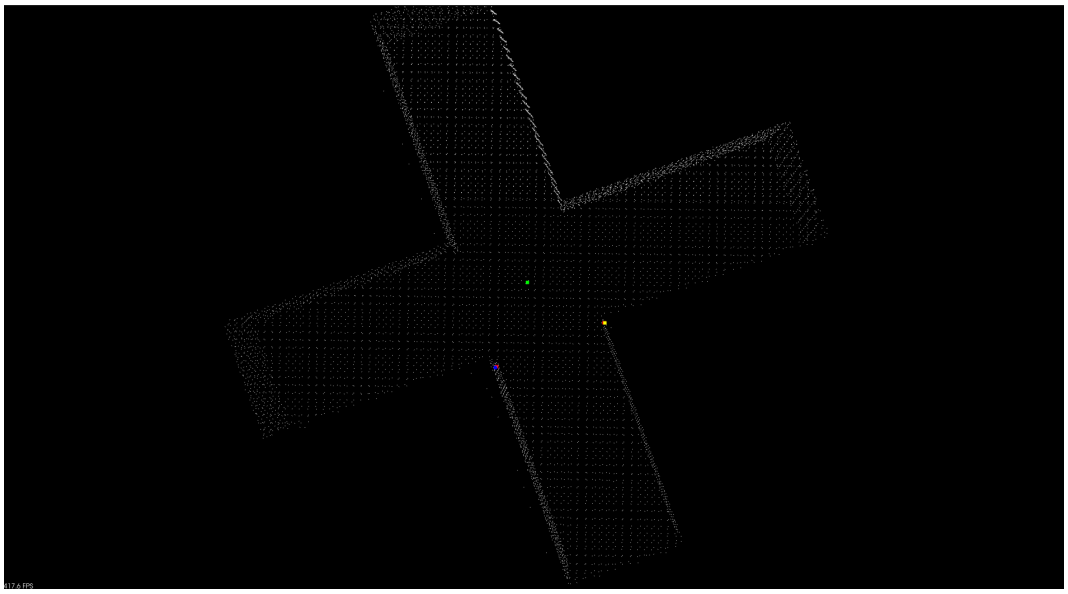

Figure 3: Cross object orientation calculation - PCL Viewer

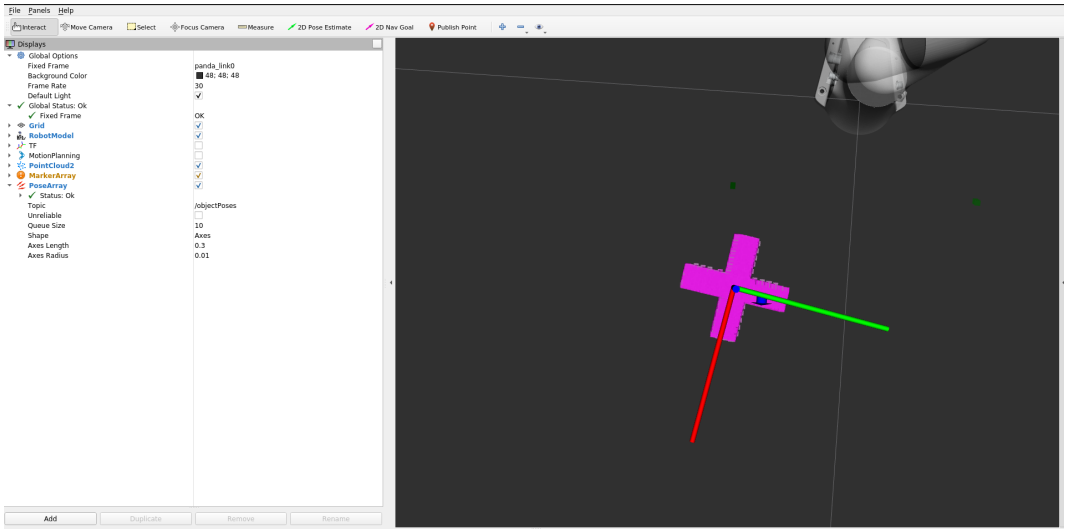

Figure 4: Orientation publish in RViz

4 Task 2

For Task 2, our approach begins with a comprehensive scan of both the front and rear sections of the working area. We then employ a multi-step classification method to differentiate between noughts and crosses:

1. We first establish an initial centroid approximation by taking a top slice of the pointcloud and calculating its average position.

2. Using this preliminary estimate as a reference point, we apply the Harris Corner detection algorithm to identify the four points closest to this estimated centroid.

3. These four points are then averaged to produce a refined and accurate centroid location for any object in the workspace.

4. Our classification logic hinges on pointcloud density near the centroid: if no points are detected near the object's centroid, we classify it as a nought; otherwise, it's identified as a cross.

5. The system outputs the classification results in the console, matching each mystery shape to its corresponding reference shape.

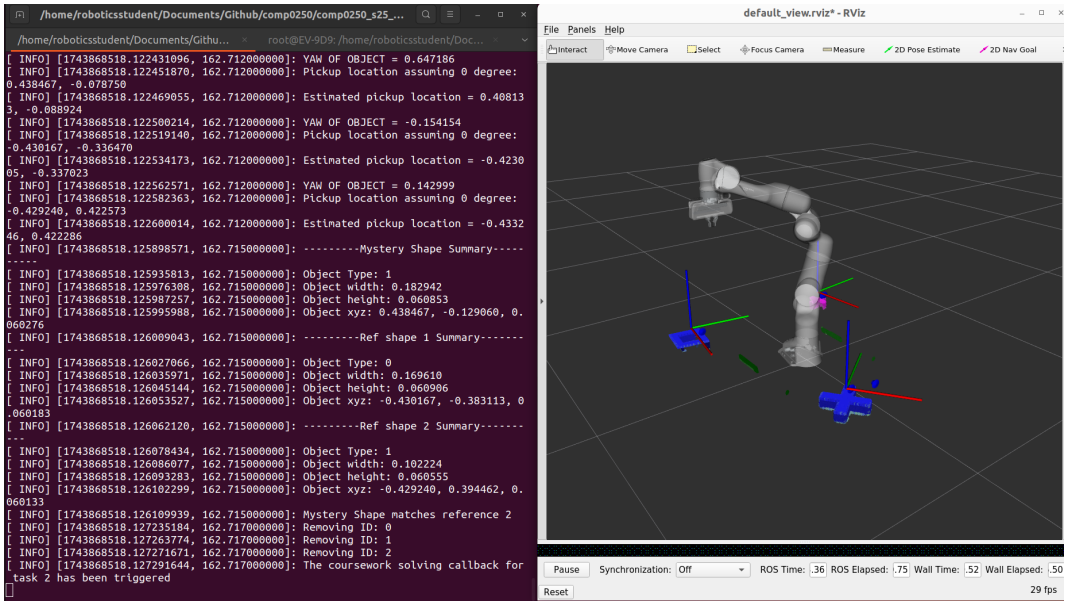

Figure 5: Task 2 - visualisation and output

5 Task 3

For Task 3 our implementation integrates the advanced techniques developed in Tasks 1 and 2 to create a comprehensive solution. The process begins with a thorough scan of the entire workspace, followed by detailed metric analysis of each detected object. Based on these metrics, we classify objects into four distinct categories: noughts, crosses, obstacles, and boxes.

This classification process is computationally intensive and may require up to 20 seconds to complete after the scanning phase. If you observe the robot appearing inactive after completing its scans, please allow this processing time before taking action. The system logs progress to the console, which you can monitor to confirm whether processing is ongoing or has stalled. In rare cases where the system has stopped responding, restarting the task will resolve the issue.

Once all objects are classified, our algorithm stores noughts and crosses in separate data structures and analyses their relative frequencies. To ensure safe operation, we select pickup candidates by calculating their Euclidean distances from all identified obstacles, prioritising objects that maximise safety margins during the manipulation process.

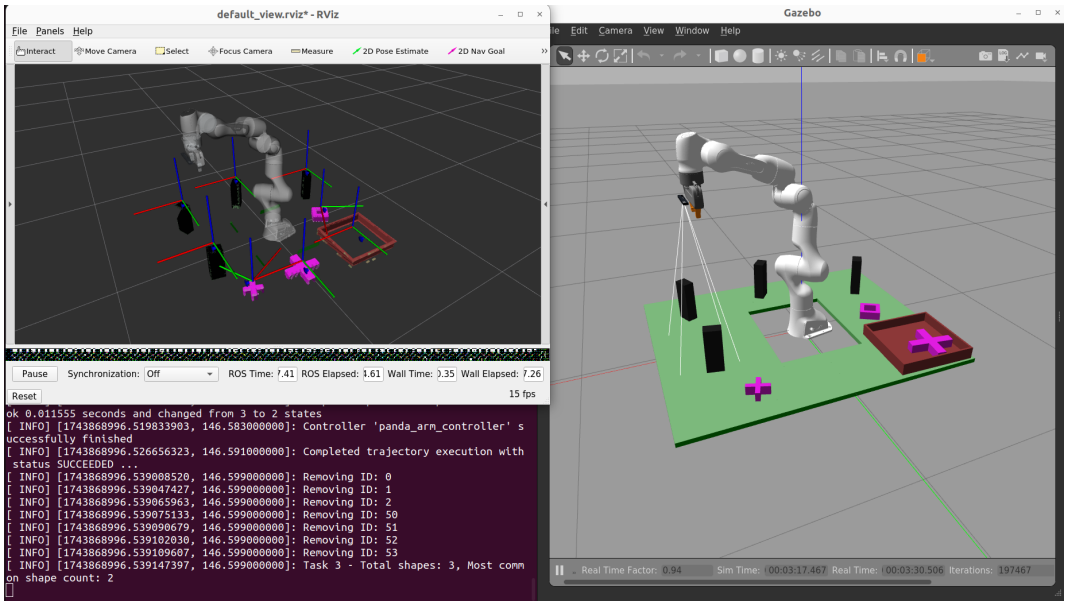

Figure 6: Task 3 - visualisation and output